DIPLOMA ELECTRONICS / ECE / BIOMEDICAL · SEMESTER 5/6/5

Embedded Systems

This is a screen-filled handbook for Course Code **5041**. It expands the syllabus into study notes, engineering explanation, diagrams, embedded C program patterns, formulas, quick revision, viva questions, model paper and answer key. It is written for exam preparation and for real electronics work where microcontrollers read inputs, drive outputs, communicate with devices and run time-critical tasks.

COURSE CODE	**5041**
CREDITS	**4**
PERIODS	**60**
PATTERN	**L:3 T:1 P:0**

COURSE OVERVIEW

What you must be able to do after this course

The syllabus objective is not only to memorize ATmega32 pins. The real target is to understand embedded architecture, select an AVR microcontroller, write embedded C, interface common peripherals, exchange data using communication interfaces and understand why RTOS is used in time-critical products.

Official course facts

Item	Details
Program	Diploma in Electronics Engineering / Electronics and Communication Engineering / Biomedical Engineering
Course code	5041
Course title	Embedded Systems
Semester	5 / 6 / 5
Credits	4
Periods	4 per week, 60 per semester

Why it matters in industry

- ✓ Every controller board has the same logic: power, clock, reset, CPU, memory, inputs, outputs and firmware.
- ✓ Escalator panels, lift display boards, HMI interface modules, sensor cards, biomedical devices and UPS controllers are embedded systems.
- ✓ Good firmware is predictable. Bad firmware randomly fails when timing, interrupts, wiring noise or communication errors are ignored.
- ✓ Real maintenance skill means reading circuit symptoms and firmware behaviour together, not separately.

□□□ □□□ theory subject □□□□ □□□□□□ □□□□□□□□ □□□□□□□□□□. Sensor input, relay output, timer delay, interrupt response, communication error □□□□□□ practical □□□ □□□□□□□□□□□□□ □□□□□□□□.

CO1

Explain embedded system basics and architecture. You must distinguish embedded systems from general computers, classify them and identify hardware/software building blocks.

CO2

Use AVR microcontrollers and embedded C. You must understand ATmega32 registers, memory, ports, timers, interrupts and write C programs for I/O and timing.

CO3

Interface peripheral devices. You must explain and realize LED, switch, relay, optocoupler, sensor, seven-segment, LCD, keyboard, motors, RTC, ADC, I2C and SPI interfacing.

CO4

Familiarize RTOS. You must know kernel functions, tasks, scheduling, communication, synchronization, drivers and RTOS selection criteria.

Prerequisite check

You need C programming, Python/programming logic and basic microcontroller architecture. Missing these basics makes Embedded C difficult because register-level firmware is unforgiving: one wrong bit can make the whole circuit appear dead.

OFFICIAL SYLLABUS MAP

Module-wise structure from the syllabus

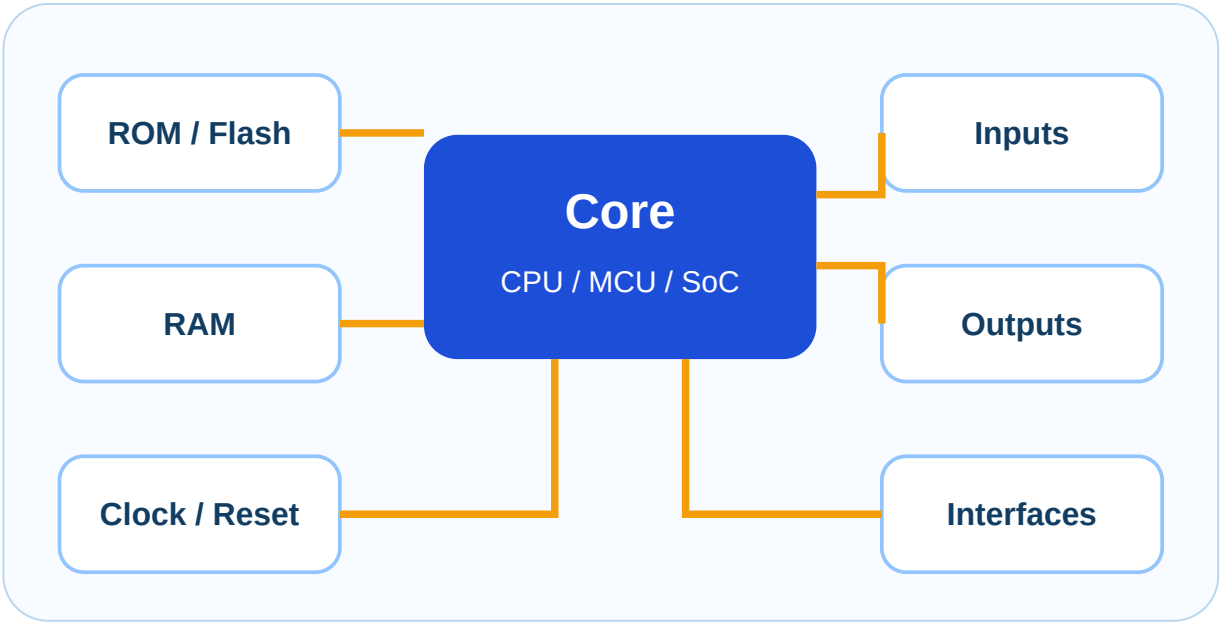
The handbook follows the uploaded PDF sequence: CO1 embedded systems, CO2 AVR and embedded C, CO3 peripheral interfacing, and CO4 RTOS.

CO	Module outcome group	Hours	Cognitive level	Core contents
CO1	Embedded system basics and architecture	13	Understanding	Definition, classification, application areas, hardware/software components, memory, sensors, actuators, I/O subsystem, onboard and external interfaces.
CO2	AVR microcontroller and embedded C	16	Applying	AVR family, ATmega32 block diagram, registers, memory organization, status register, program counter, I/O ports, timers, interrupts, embedded C programs.

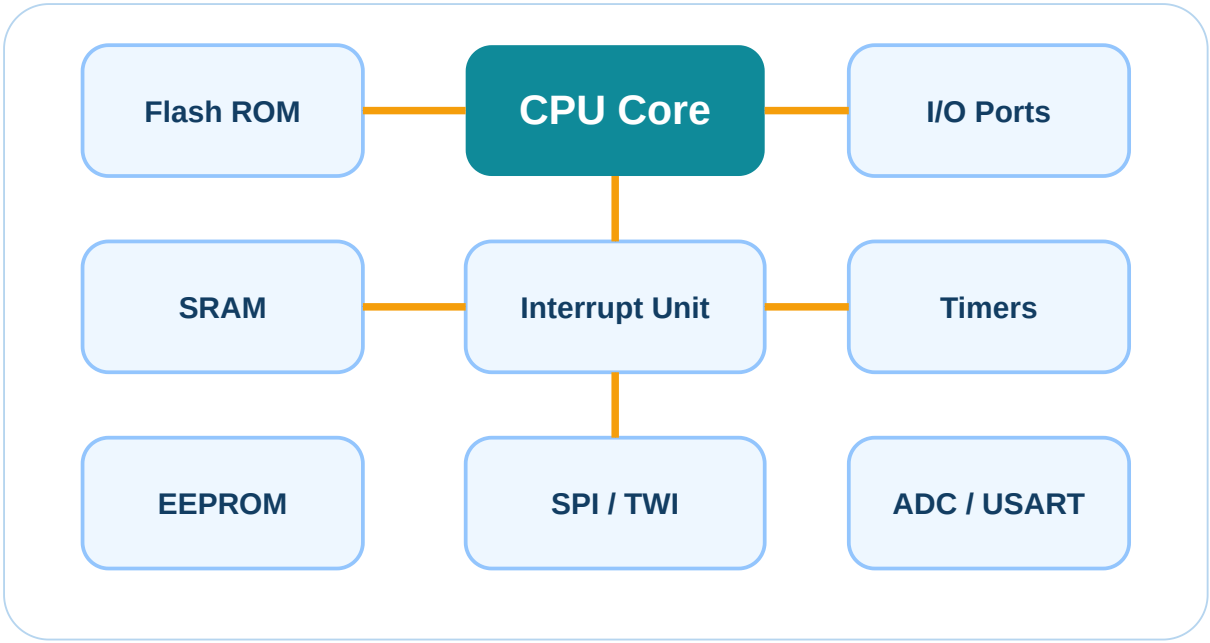
CO3	Peripheral interfacing with AVR	19	Applying	LED, switch, relay, optocoupler, sensors, seven segment, LCD, keyboard, DC/servo/stepper motor, RTC, ADC, I2C and SPI.
CO4	RTOS fundamentals	10	Understanding	Kernel, GPOS, RTOS, tasks, processes, threads, scheduling, task communication, synchronization, drivers, selection criteria and Micro C/OS-II overview.

VISUAL FOUNDATION

Architecture diagrams you should be able to redraw



Basic embedded-system building blocks.



ATmega32 simplified block view for memory, peripherals and interrupts.

MODULE 1 · CO1 · 13 HOURS

Embedded Systems: Definition, classification, architecture and interfaces

Goal: explain what makes a system embedded, identify its blocks and relate each block to real products.

1. Definition

An embedded system is a dedicated electronic system in which hardware and software are designed together to perform one or a limited set of functions. Unlike a laptop, the user normally does not install general applications. The firmware is stored in non-volatile memory and runs when power is applied.

> Dedicated task □□□□□□□ hardware + firmware □□□□□□□ □□□□□□□□□□□□ system □□□ embedded system.

2. Difference from general purpose computer

- Dedicated function, limited user interface.
- Firmware stored in ROM/Flash.
- Often real-time: response must happen within a time limit.
- Lower cost, lower power and smaller size.
- Direct connection to sensors, actuators and communication buses.

3. Classification

Small scale: 8-bit/16-bit MCU, simple firmware. **Medium scale:** 16/32-bit controller, RTOS possible. **Sophisticated:** high performance SoC, OS/RTOS, networking. **Real-time:** hard, firm or soft time constraints.

4. Application areas

Escalator/lift controllers, vehicle ECU, washing machine, medical monitor, ECG acquisition, smart energy meter, inverter, UPS, printer, router, camera, fire alarm panel and industrial HMI modules.

5. Hardware components

Microcontroller, clock oscillator, reset circuit, power supply, memory, input conditioning, output driver, communication interface, display, sensors, actuators and protection circuits.

6. Software components

Startup code, drivers, main application loop, interrupt service routines, communication stack, fault handler, configuration data, test routines and in advanced systems an RTOS.

7. Memory

ROM/Flash: stores program. **RAM/SRAM:** temporary variables and stack. **EEPROM:** stores calibration or settings. Memory errors are serious because firmware depends on exact addresses and bits.

8. Sensors, actuators and I/O

Sensors convert physical quantities into electrical signals. Actuators convert electrical signals into physical action. I/O subsystem is the bridge between the microcontroller and external world.

9. On-board interfaces

I2C, SPI and UART are used between devices on the same board or within a product. I2C uses address-based two-wire communication, SPI uses fast master-slave clocked transfer, and UART sends asynchronous serial data.

10. External interfaces

RS-232, USB, Bluetooth and Wi-Fi are used to connect outside modules, computers or networked devices. In real panels, isolation, grounding and cable routing are as important as protocol logic.

Exam trap: Students often write only examples like washing machine and skip architecture. For full marks, write definition, differences, classification, blocks and applications.

MODULE 2 · CO2 · 16 HOURS

AVR microcontroller, ATmega32 architecture and Embedded C

Goal: understand ATmega32 blocks, registers, I/O ports, timers and interrupts; then convert that knowledge into embedded C programs.

AVR family and microcontroller selection

AVR microcontrollers are RISC-based controllers widely used for small and medium embedded products. Selection depends on program memory size, RAM, number of I/O pins, ADC channels, timers, communication interfaces, operating voltage, package, speed, cost and availability.

Criterion	Why it matters
I/O pins	Needed for LED, switch, relay, display, keyboard, motor driver and sensors.
Flash memory	Must fit firmware code.

SRAM	Required for variables, buffers and stack.
ADC	Needed for analog sensors like temperature.
Timers	Needed for delay, PWM, event counting and scheduling.
Interfaces	UART, SPI and I2C decide communication ability.

ATmega32 important blocks

- ✓ 8-bit AVR CPU core
- ✓ 32 KB Flash program memory
- ✓ SRAM and EEPROM data memory
- ✓ General purpose registers and SFRs
- ✓ Four 8-bit I/O ports: PORTA, PORTB, PORTC, PORTD
- ✓ Timers/counters 0, 1 and 2
- ✓ ADC, USART, SPI and TWI/I2C
- ✓ Interrupt controller, watchdog timer and power modes

Status Register SREG

SREG stores flags produced by arithmetic and logic operations. Common flags: C carry, Z zero, N negative, V overflow, S sign, H half-carry, T bit copy storage and I global interrupt enable.

Program Counter

The program counter points to the next instruction in Flash memory. During jumps, calls and interrupts, its value changes so execution continues from a different location.

I/O port registers

DDRx sets input/output direction. **PORTx** writes output or enables pull-up on input. **PINx** reads actual pin level.

Timer concept

A timer counts clock pulses. With a prescaler, the timer clock is reduced. When the counter overflows or matches a compare value, a flag or interrupt can be generated.

Timer tick time = Prescaler / F_{CPU}

Overflow time for 8-bit timer = (256 − initial count) × tick time

Delay loop □□□□□□□□□□□□□□□□ crude method □□□. Timer □□□□□□□□□□□ delay predictable □□□□.

Interrupt concept

An interrupt temporarily stops the main program and executes an ISR when an important event occurs. It is used for button events, timer overflow, ADC completion, UART receive and other time-sensitive events. Keep ISR short; long ISR creates timing problems.

1. Enable global interrupt.
2. Enable selected peripheral interrupt.
3. Event occurs and flag is set.
4. CPU jumps to ISR vector.
5. ISR handles event and returns.

MODULE 3 · CO3 · 19 HOURS

Peripheral interfacing with AVR

Goal: connect real-world devices to AVR and understand the role of driver circuits, pull-ups, current limiters, protection and communication protocols.

LED interfacing

An LED must be connected with a current limiting resistor. The port pin can source or sink current within its rating. Never connect an LED directly to a microcontroller pin.

$R = (V_{CC} - V_F) / I_{LED}$

Push button interfacing

A switch input needs a defined logic level. Use pull-up or pull-down resistor. Software debounce is needed because mechanical contacts bounce.

Relay and optocoupler

A relay needs a transistor driver and flyback diode. An optocoupler provides electrical isolation between control side and high-noise/load side.

Temperature and IR sensors

Analog sensors need ADC; digital sensors may use logic inputs or serial protocols. In industrial wiring, sensor filtering and shielding prevent false triggering.

Seven segment display

Segments are LEDs arranged as a digit. Common anode and common cathode types require opposite drive logic. Multiplexing reduces I/O pins for multiple digits.

16x2 LCD

LCD modules commonly use 4-bit or 8-bit parallel interface. Important pins: RS, RW, EN and data lines. 4-bit mode saves pins but needs two nibbles per character.

Keyboard interfacing

A 4x4 matrix keypad has 4 rows and 4 columns. Scan one row/column at a time and read the other side to identify the pressed key.

Motor interfacing

DC motor requires a driver transistor/H-bridge. Servo needs PWM signal. Stepper motor needs sequential phase excitation through driver IC/transistors.

RTC and I2C

Real Time Clock chips commonly use I2C. I2C has SDA and SCL lines with pull-up resistors. Each device has an address.

SPI

SPI uses MOSI, MISO, SCK and SS. It is faster than I2C and commonly used for displays, sensors, memory cards and ADC/DAC ICs.

ADC interfacing

ADC converts analog voltage into digital number. Resolution decides step size. For 10-bit ADC with 5 V reference, one count is about 4.88 mV.

> $\text{ADC count} = V_{in} \times 1024 / V_{ref}$

LM35 temperature sensor

LM35 output is approximately 10 mV per °C. If ADC reference is 5 V and ADC is 10-bit, temperature can be calculated from ADC count.

> $\text{Temperature °C} \approx \text{ADC} \times 500 / 1024$

MODULE 4 · CO4 · 10 HOURS

RTOS: kernel, tasks, scheduling, communication and selection

Goal: understand why real-time operating systems are used when a simple super-loop becomes weak.

OS basics

An operating system manages hardware and software resources. A kernel is the core part that handles scheduling, memory, interrupts, timers, task management and device drivers.

GPOS: optimized for general use and throughput. **RTOS:** optimized for deterministic response time.

Task, process and thread

A task is an executable unit in RTOS. A process is a program in execution with its own resources. A thread is a lightweight execution path. Embedded RTOS commonly uses tasks/threads with priorities.

RTOS kernel functions

- Task creation and deletion
- Scheduling and context switching
- Delay and time management
- Interrupt handling support
- Semaphore and mutex services
- Queues and mailboxes
- Device driver support

Scheduling types

Pre-emptive: higher priority task interrupts lower priority task. **Co-operative:** task voluntarily gives CPU. **Round robin:** equal priority tasks share CPU time.

Synchronization

Synchronization avoids data corruption when multiple tasks or interrupts share resources. Common tools: semaphore, mutex, event flag, queue and critical section.

RTOS selection criteria

- ✓ Deterministic scheduling and interrupt latency
- ✓ Memory footprint suitable for MCU
- ✓ Supported CPU/compiler/toolchain
- ✓ Driver and communication stack support
- ✓ Licensing and cost
- ✓ Documentation and community
- ✓ Safety/reliability requirement

Popular RTOS examples

FreeRTOS, Micro C/OS-II, Zephyr, VxWorks, QNX, ThreadX/Azure RTOS, RTEMS, NuttX, embOS, Integrity, SafeRTOS and TI-RTOS.

> RTOS □□□□□□□ fast □□□□□□□ □□□□□□□ □□□□□. Predictable timing □□□□□□□□ □□□□ □□□□□□□□□□□ □□□□□□□□□□□□□□□□□.

PROGRAM BANK

Embedded C patterns for ATmega32

These snippets are study patterns. Before using real hardware, check exact crystal frequency, board schematic, header file, fuse settings, programmer settings and pin current limits.

1. LED ON/OFF using PORTB

```
#include <avr/io.h>
#define F_CPU 8000000UL
#include <util/delay.h>

int main(void){
    DDRB |= (1<<PB0);        // PB0 as output
    while(1){
        PORTB ^= (1<<PB0);   // Toggle LED
        _delay_ms(500);
    }
}
```

2. Read push button with pull-up

```
#include <avr/io.h>
int main(void){
    DDRD &= ~(1<<PD2);       // PD2 input
    PORTD |= (1<<PD2);       // Enable pull-up
    DDRB |= (1<<PB0);        // LED output
    while(1){
        if(!(PIND & (1<<PD2))) PORTB |= (1<<PB0);
        else PORTB &= ~(1<<PB0);
```

```
    }
}
```

3. ADC read idea

```
void adc_init(){
    ADMUX = (1<<REFS0);      // AVCC reference
    ADCSRA = (1<<ADEN) | (1<<ADPS2) | (1<<ADPS1);
}
unsigned int adc_read(unsigned char ch){
    ADMUX = (ADMUX & 0xF0) | (ch & 0x0F);
    ADCSRA |= (1<<ADSC);
    while(ADCSRA & (1<<ADSC));
    return ADC;
}
```

4. SPI transfer idea

```
void spi_init_master(){
    DDRB |= (1<<PB5)|(1<<PB7)|(1<<PB4); // MOSI,SCK,SS
    DDRB &= ~(1<<PB6);                  // MISO
    SPCR = (1<<SPE)|(1<<MSTR)|(1<<SPR0);
}
unsigned char spi_tx(unsigned char data){
    SPDR = data;
    while(!(SPSR & (1<<SPIF)));
    return SPDR;
}
```

Program-writing discipline

- ✓ First configure direction register, then write/read port.
- ✓ Always calculate timer values from F_CPU and prescaler; do not guess.
- ✓ Use current limiting resistor and driver transistor for external loads.
- ✓ Do not connect relay or motor directly to MCU pin.
- ✓ For I2C, check pull-up resistors and device address.

✓ For RTOS, never put long blocking delays inside high-priority tasks.

FORMULA BANK

Calculations used in embedded systems

LED resistor

$$R = (V_{CC} - V_F) / I$$

Example: 5 V supply, 2 V LED, 10 mA current → R = 300 Ω. Use nearest standard value above.

ADC resolution

$$\text{Step size} = V_{ref} / 2^n$$

10-bit ADC, 5 V reference → 5/1024 = 4.88 mV/count.

LM35 conversion

$$\text{Temp °C} = V_{out} / 10\ \text{mV}$$

If Vout = 300 mV, temperature = 30 °C.

Timer tick

$$T_{tick} = \text{Prescaler} / F_{CPU}$$

Timer overflow

$$T = (\text{Max count} - \text{Initial count}) \times T_{tick}$$

UART frame time

> Time per bit = 1 / Baud rate

At 9600 baud, one bit ≈ 104.17 µs.

VIVA AND TROUBLESHOOTING

Questions that expose weak understanding

Why is a resistor required with LED?

Because LED current rises rapidly after forward voltage. Without a resistor, excessive current can damage LED or MCU pin.

Why is pull-up required for switch input?

A floating input can randomly read 0 or 1 due to noise. Pull-up gives a defined HIGH when switch is open.

Why should ISR be short?

Long ISR blocks other interrupts and disturbs system timing. ISR should set flags and exit quickly.

Why use optocoupler?

For electrical isolation between low-voltage controller and noisy/high-voltage external circuit.

ADC reading is fluctuating. What to check?

Check reference voltage, sensor wiring, grounding, analog filtering, conversion clock, shielding and whether input impedance is suitable.

I2C device is not responding. What to check?

Check SDA/SCL pull-ups, correct address, common ground, clock speed, bus short, power supply and whether another device holds the bus low.

Relay chatters when motor starts. Why?

Possible supply dip, poor grounding, missing flyback diode, noise coupling, weak driver or firmware not debounced.

Why RTOS instead of while loop?

Use RTOS when multiple time-critical tasks need predictable scheduling, synchronization and communication.

PRACTICE

Module-wise question bank

M1-01 Define embedded system and compare it with a general purpose computer.

M1-02 Classify embedded systems with examples.

M1-03 Draw and explain the building blocks of a typical embedded system.

M1-04 Explain ROM, RAM, sensors, actuators and I/O subsystem in embedded architecture.

M1-05 Differentiate onboard and external communication interfaces.

M2-01 List the criteria for selecting a microcontroller for an embedded application.

M2-02 Explain the simplified block diagram of ATmega32.

M2-03 Explain SREG, program counter, I/O registers and memory organization in AVR.

M2-04 Write an embedded C program to blink an LED connected to an AVR port.

M2-05 Explain timer delay generation using prescaler and overflow.

M2-06 Explain external hardware interrupt programming steps.

M3-01 Explain LED and push button interfacing with AVR.

M3-02 Explain relay and optocoupler interfacing and state why driver circuits are required.

M3-03 Explain ADC interfacing and derive temperature from LM35 output.

M3-04 Explain seven segment and LCD interfacing.

M3-05 Compare DC motor, servo motor and stepper motor interfacing.

M3-06 Explain I2C RTC interfacing and basics of SPI communication.

M4-01 Differentiate GPOS and RTOS.

M4-02 Explain task, process, thread, scheduling and synchronization.

M4-03 List RTOS selection criteria and popular RTOS examples.

MODEL QUESTION PAPER

Embedded Systems 5041 · Practice paper with answer key

This is an original practice paper following the syllabus structure. It is not a copied official question paper.

PART A - Short Answer

1. Define embedded system.
2. List any four application areas of embedded systems.
3. What is the use of SREG in AVR?
4. What are DDRx, PORTx and PINx registers?
5. State the purpose of timer prescaler.
6. Why is a relay driver required?
7. What is I2C?
8. What is SPI?
9. Define RTOS.
10. List any two RTOS synchronization methods.

PART B - Short Essay

11. Compare embedded system and general purpose computer.
12. Draw and explain the building blocks of an embedded system.
13. Explain the simplified block diagram of ATmega32.
14. Explain AVR memory organization and I/O port registers.
15. Write an embedded C program to blink an LED and explain each line.
16. Explain timer delay calculation in AVR.
17. Explain LED, push button, relay and optocoupler interfacing.
18. Explain ADC interfacing with LM35 temperature sensor.
19. Explain I2C RTC interfacing and SPI basics.
20. Explain kernel functions and task scheduling in RTOS.

PART C - Long Essay

21. Explain AVR interrupts, timer interrupts and external hardware interrupts with programming steps.
22. Explain LCD, keyboard, DC motor, servo motor and stepper motor interfacing with AVR.

23. Explain GPOS, RTOS, tasks, processes, threads, task communication, synchronization, device drivers and RTOS selection criteria.

ANSWER KEY

Compact answers for model paper

Part A key

1. Dedicated hardware-software system for a specific function.
2. Washing machine, escalator controller, ECG monitor, smart meter, router.
3. SREG stores CPU status flags such as zero, carry and interrupt enable.
4. DDRx selects direction, PORTx writes output/enables pull-up, PINx reads pin.
5. Prescaler divides CPU clock to make timer tick slower.
6. MCU pin cannot supply relay coil current and needs isolation/protection.
7. Two-wire serial bus using SDA and SCL with device addressing.
8. Synchronous serial interface using MOSI, MISO, SCK and SS.
9. Operating system designed for predictable response within time limits.
10. Semaphore, mutex, queue, event flag.

Part B/C answer direction

For long answers, draw neat block diagrams first, then explain block function, register role, signal flow, timing and applications. For interfacing questions, always mention current limiting, driver, isolation, pull-up/down, firmware logic and protection. For RTOS, explain deterministic timing and priority scheduling, not just names of operating systems.

> Full mark answer: diagram + working + register/function + practical point + application. □□□□□□□ □□□□□□□□ answer standard □□□□.

REFERENCES

Textbooks and online resources from syllabus

Type	Source
T1	K. V. Shibu, Introduction to Embedded Systems, 2e, McGraw Hill Education India, 2016.
T2	Rajkamal, Embedded Systems Architecture, Programming and Design, TMH, 2003.

R1	Muhammad Ali Mazidi, Sarmad Naimi and Sepehr Naimi, The AVR Microcontroller and Embedded Systems Using Assembly and C, Pearson Education.
R2	Michael J. Pont, Embedded C, Pearson Education, Second Edition.
Online	Embedded programming tutorials, Tutorialspoint Embedded Systems, EmbeddedSchool AVR programming, SparkFun I2C, ElectronicWings AVR I2C/SPI and Intel MicroC/OS-II overview.